Graph-Based Smart Contract Auditing: A Leave-One-Contract-Out Evaluation of a Graph Attention Network Prototype

[Student Name] · Student ID [Student ID]
Final project report · October 2026

Abstract

Smart contracts on public blockchains are immutable once deployed, so a code-level flaw can cause irreversible loss. We present a pre-deployment auditing prototype for Solidity. It compiles a contract, builds a multi-relational graph from its abstract syntax tree, control-flow graph and data-flow edges, slices the graph around security-relevant statements, and classifies it with a Graph Attention Network (GAT). The model gives a contract-level verdict and a node-level score that points to lines. The prototype covers three vulnerability classes: Reentrancy, Timestamp Dependency and Arithmetic Error. We evaluate it on six hand-written demo contracts, one vulnerable and one safe per class. The model fits its training contracts perfectly (18 of 18 class decisions correct). Under leave-one-contract-out evaluation, however, it flags none of the three held-out vulnerable contracts across five seeds (recall 0.000). A rule that flags any contract containing the relevant sink reaches recall 1.000 with precision 0.429. Logistic regression on pooled node features predicts "safe" throughout. We also found and fixed two code defects: training labels for two contracts that pointed at the wrong lines, and an arithmetic operator lookup that failed for legacy-format syntax trees. The results show that the pipeline runs end to end, that its in-sample fit does not generalize at this scale, and that a larger labelled benchmark is needed before any detection claim can be made.

Keywords: smart contracts; Solidity; graph attention network; program slicing; vulnerability detection; leave-one-out evaluation

1. Introduction

Smart contracts are programs that hold and move digital assets on blockchains such as Ethereum. Once deployed, the code cannot be patched, so a flaw can be exploited before its owner notices (Zheng et al., 2020). Two classes of flaw illustrate the stakes. Reentrancy lets a callee re-enter a function before the caller updates its state; this is the mechanism behind the 2016 TheDAO loss (Luu et al., 2016). Timestamp dependence lets a miner influence logic that relies on `block.timestamp`. Arithmetic errors wrap integers silently in pre-0.8 Solidity.

Automated tools address these flaws with symbolic execution, pattern matching or static analysis. Oyente runs symbolic execution on bytecode (Luu et al., 2016), SmartCheck checks Solidity source with extensible rules (Tikhomirov et al., 2018), and Slither lifts Solidity into an SSA intermediate representation (Feist et al., 2019). A large empirical review found that these tools disagree with each other, and that a high share of real contracts is flagged (Durieux et al., 2020). Learning-based

detectors have been proposed as a way to reduce false alarms (De Baets et al., 2024), and graph neural networks are the most common choice because a contract's control and data flow is naturally a graph (Zhuang et al., 2020; Liu et al., 2023).

This project builds such a detector and asks three questions.

- **RQ1.** Can a GAT over a sliced contract graph learn to separate vulnerable from safe contracts for the three classes?
- **RQ2.** Does it generalize to a contract it has not seen, and how does it compare with simple baselines under the same protocol?
- **RQ3.** Can its node-level scores localize the vulnerable line?

The contribution is the working pipeline, a leave-one-contract-out protocol with two baselines, a documented set of defects found and fixed during the work, and a negative result on generalization at this scale. We do not claim a detection rate.

2. Related work

Graph attention. The GAT layer computes attention coefficients over a node's neighbours and aggregates their features with those weights (Veličković et al., 2018). Graph convolution with a first-order spectral approximation is the simpler alternative (Kipf & Welling, 2017). The classifier here uses the GAT form.

Graph-based contract detection. Zhuang et al. (2020) build a graph of each function's syntactic and semantic structure, reduce it with an elimination step and apply a graph convolution with temporal message propagation. Liu et al. (2023) model contracts as a nested multi-relational graph of functions and localize vulnerable functions. Nguyen et al. (2022) use heterogeneous graph embeddings with metapaths for fine-grained detection. These methods are trained and tested on large corpora. The present work uses six contracts and is therefore a feasibility study.

Tool evaluation. Durieux et al. (2020) ran nine tools on 47,518 contracts and 69 annotated vulnerable contracts, and reported low agreement between tools and a high flag rate on real contracts. Ghaleb and Pattabiraman (2020) evaluated six tools by injecting bugs into contracts (SolidiFI). SolidiFI and the SmartBugs collection are natural held-out benchmarks for future work (Section 8). Sayeed et al. (2020) reviewed attacks and reported that the tools they tested still miss known flaws.

The earlier draft of this work cited six further papers that could not be verified against a publisher or arXiv record. They were removed. The verified literature is listed in `LITERATURE_MATRIX.md` in the repository.

3. Data and ethics

The data are six Solidity files written for this project (Table 1). Each vulnerable file contains one minimal instance of its pattern. Each safe file is a counterpart with the pattern removed or fixed: a state update before the external call for Reentrancy, no timestamp dependence for Timestamp, and Solidity 0.8 (checked arithmetic) for Arithmetic. The Timestamp safe file is not a line-for-line

variant of the vulnerable one. The files are not copies of deployed code. Labels are the author's annotations, checked by reading each file. The data card is `DATA_CARD.md`.

No human subjects, personal data, secrets or live-network queries are involved. The vulnerable examples are the classic public patterns in minimal form and contain no working exploit. The ethics statement is `ETHICS.md`.

Table 1. Demo contracts and labels. Line numbers were checked against the files.

Contract	Compiler	Class	Label	Vulnerable line(s)
ReentrancyVulnerable.sol	0.8.x	Reentrancy	vulnerable	15, 16
ReentrancySafe.sol	0.8.x	Reentrancy	safe	none
TimestampVulnerable.sol	0.8.x	Timestamp	vulnerable	9
TimestampSafe.sol	0.8.x	Timestamp	safe	none
ArithmeticVulnerable.sol	0.7.6	Arithmetic	vulnerable	12
ArithmeticSafe.sol	0.8.x	Arithmetic	safe	none

4. Method

4.1 Graph construction

The compiler wrapper (`parser/compiler.py`) reads the pragma, selects a solc version (0.7.6 or 0.8.20 in these experiments) and returns the AST. The graph builder (`parser/graph_builder.py`) creates a node for each AST node with its line and source text, an AST edge for each parent-child pair, control-flow edges from a statement walk, and data-flow edges from identifier references, assignments, declarations and call arguments. Both the legacy AST (solc 0.7.x, nodes labelled by `name` with children in `children`) and the compact AST (solc 0.8.x, `nodeType` with named fields) are handled.

4.2 Slicing

Interest points are external calls (`.call`, `transfer`, `send`), references to `block.timestamp` or `now`, and binary operators `+ - * /`. From each interest point, the slicer follows control and data edges forward and backward for three steps, then adds up to three AST ancestors so the slice remains a connected tree (`slicing/slicer.py`). If a contract has no interest point, the seeds are function calls and variable declarations.

4.3 Model

Each node has a 25-dimensional feature vector: a one-hot code for its AST type over 20 types plus an "other" bin, and four keyword flags (call, timestamp, arithmetic, state write). Two GAT layers with hidden size 64 and a dropout of 0.1 produce node embeddings. A node head gives three per-class logits. A graph head applies to the mean of the node embeddings and gives three per-class logits. The graph loss is binary cross-entropy on the graph head, plus twice the same loss on the node head.

Training uses Adam with learning rate 0.005 and weight decay 1e-4, for 100 epochs with one optimizer step per graph. The seed is 42 for the saved weights. A class is flagged when its graph probability exceeds 0.5. A line is localized when a node in that line has a class probability above 0.3. Both thresholds were set by hand and were not tuned.

4.4 Evaluation protocol

With six contracts, a random split would leave no test positive for some classes. We use leave-one-contract-out (LOCO). For each held-out contract, the GAT is retrained on the other five and produces a verdict for each of the three classes. This gives 18 binary decisions per method (6 contracts × 3 classes), of which 3 are positive. The GAT is run with five seeds (0 to 4) to measure the effect of initialization and dropout.

Two baselines use the same folds:

- **Sink-presence rule.** Flags a class whenever the full graph contains any sink for it. It uses the same keywords as the slicer and needs no training. It is the simplest rule that tests whether the learned model adds anything.
- **Logistic regression** on the mean and maximum of the 25 node features over the sliced graph (50 values). It uses pooled features with no message passing, and tests whether graph structure is needed.

The in-sample result, which reuses the six training contracts, is also reported for comparison with the earlier draft. It is not a generalization result.

5. Results

5.1 Graph size after slicing

Table 2 lists node counts before and after slicing. The arithmetic vulnerable contract is reduced to 8 nodes (89.0%). This fits the slicer's design, but it leaves little context (see Section 6).

Table 2. Nodes before and after the 3-step slice (`results/slicing_sizes.csv`).

Contract	Full graph	Sliced graph	Reduction
ReentrancyVulnerable.sol	59	23	61.0%
ReentrancySafe.sol	59	25	57.6%
TimestampVulnerable.sol	26	13	50.0%
TimestampSafe.sol	25	19	24.0%
ArithmeticVulnerable.sol	73	8	89.0%
ArithmeticSafe.sol	73	20	72.6%

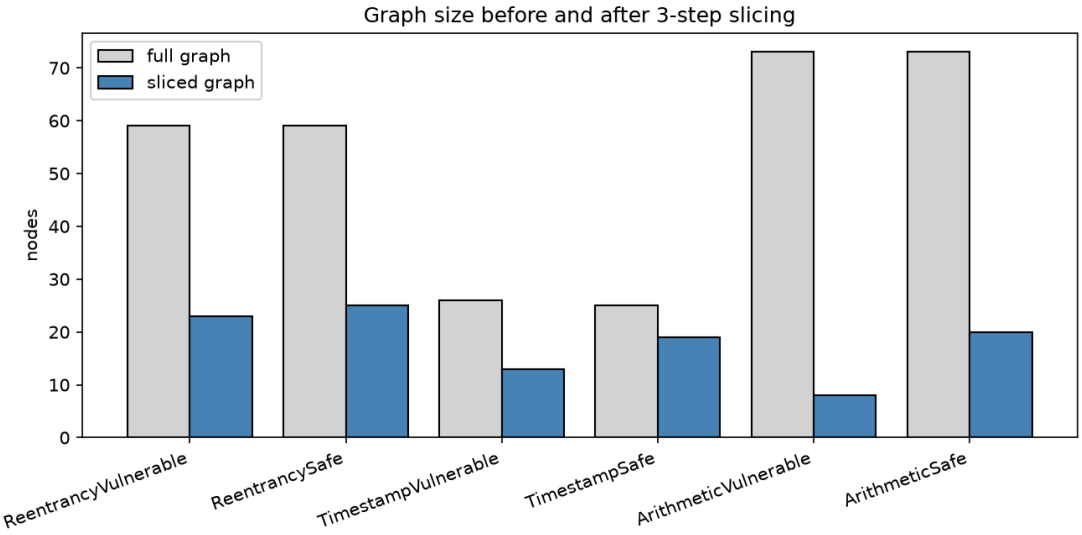


Figure 1. Node counts before and after slicing for the six demo contracts.

5.2 In-sample fit

Trained on all six contracts, the saved model gives 18 of 18 correct decisions (precision, recall and F1 all 1.000). Its confidences for the vulnerable contracts are 99.21% (Reentrancy), 99.99% (Timestamp) and 98.25% (Arithmetic). The highest confidence for any safe contract is 0.37%. This shows that the classifier can fit its training data. It says nothing about unseen contracts.

5.3 Leave-one-contract-out comparison

Table 3 gives the main result. The GAT is shown as the mean over five seeds with the standard deviation.

Table 3. Leave-one-contract-out results, 18 class decisions per method (3 positive). Source: `results/metrics.json`.

Method	TP	FP	TN	FN	Precision	Recall	F1	Accuracy
GAT (mean of 5 seeds)	0	1–2	13–14	3	0.000	0.000	0.000	0.767 ± 0.022
Sink-presence rule	3	4	11	0	0.429	1.000	0.600	0.778
Logistic regression	0	0	15	3	0.000	0.000	0.000	0.833
GAT, in-sample (saved weights)	3	0	15	0	1.000	1.000	1.000	1.000

The GAT never flags a held-out vulnerable contract. Its accuracy of 0.767 comes from correctly predicting "safe" on most decisions, which the class imbalance rewards. The sink-presence rule has perfect recall and the best F1 of the three held-out methods, but it has four false positives. Logistic regression predicts "safe" for every decision, so its accuracy of 0.833 reflects the imbalance and not detection.

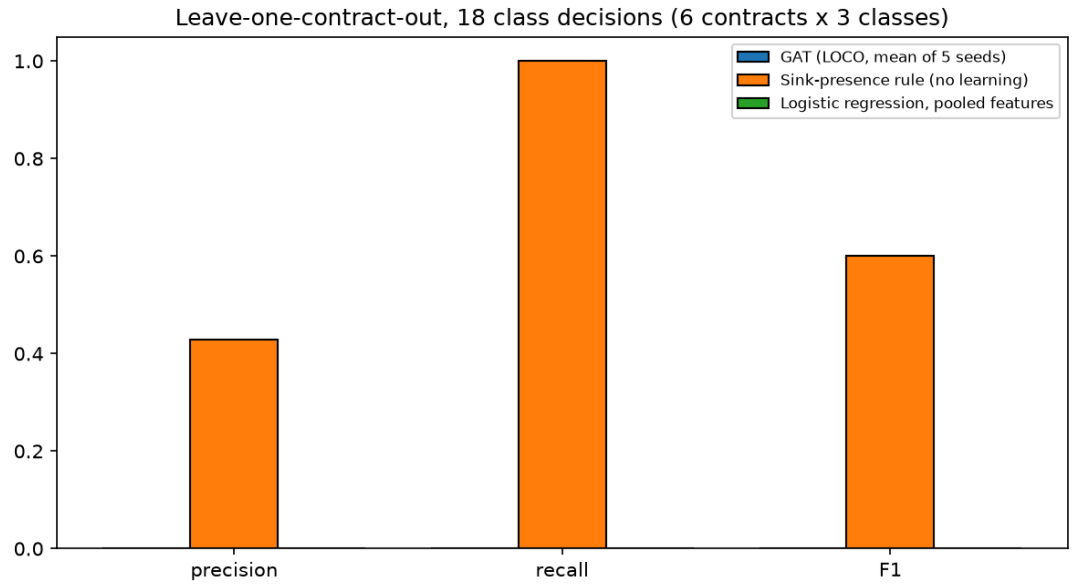


Figure 2. Leave-one-contract-out precision, recall and F1 for the three methods (18 decisions each).

5.4 Node-level localization

For the saved, in-sample model, the auditor localizes Reentrancy to lines 15 and 16 (Figure 3), which are the external call and its result check. The Timestamp contract is localized to line 9, the `require` on `block.timestamp`. The Arithmetic contract is flagged at 98.25% but no line exceeds the 0.3 node threshold, so the auditor reports that it cannot localize it.

Under leave-one-contract-out, localization fails in all 15 vulnerable-contract evaluations (3 contracts × 5 seeds): no node reaches the threshold for the held-out contract's class. The mean precision and recall are both 0.000.

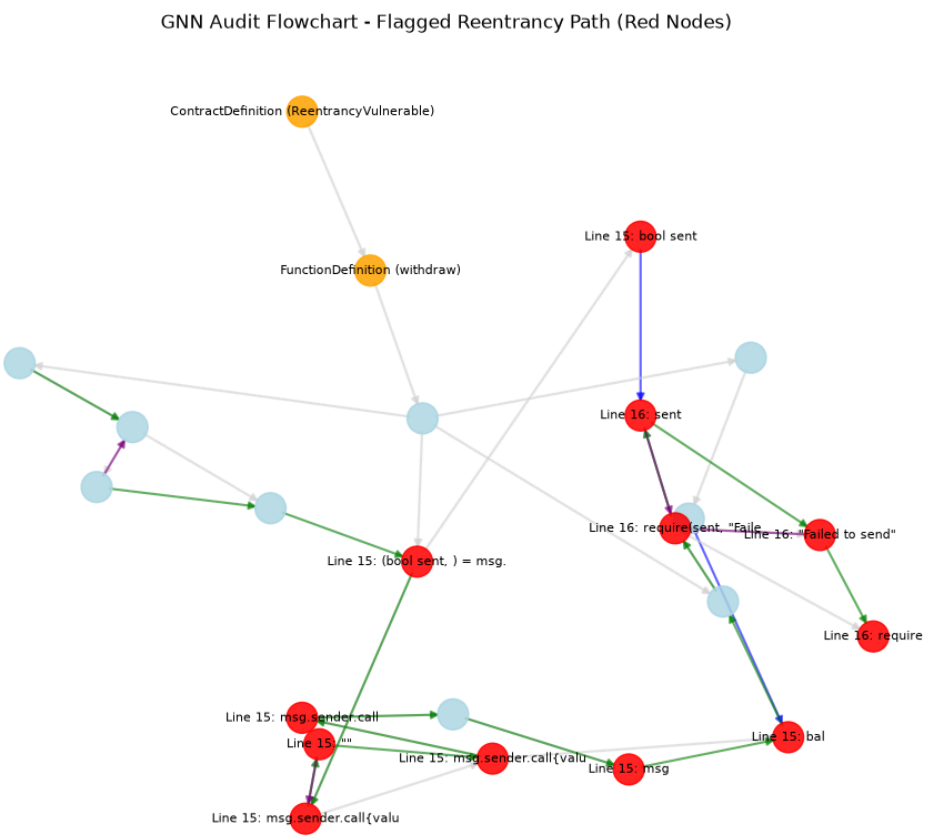


Figure 3. Output of `auditor.py --visualize` for ReentrancyVulnerable.sol (saved weights). Red nodes exceed the 0.3 threshold for the Reentrancy class and correspond to lines 15 and 16. Node positions use a spring layout with a fixed seed.

6. Error analysis

Table 4 lists the errors that matter, with the reason we found for each.

Table 4. Errors under leave-one-contract-out and their causes.

Method	Contract and class	Error	Cause
GAT (all seeds)	ReentrancyVulnerable, Reentrancy	FN, probability ~3e-6 (seed 0)	When it is held out, the only Reentrancy training example is the safe contract. A single negative example likely teaches the model to score this pattern as safe.
GAT (all seeds)	ReentrancySafe, Reentrancy	FP, probability 0.96 (seed 0)	When it is held out, the only positive example is the vulnerable contract. Both contracts contain the same external call, so a likely cause is that the model keys on the call and not on its position relative to the state update. We did not test this.
GAT (all seeds)	TimestampVulnerable, Timestamp	FN in all seeds	The only timestamp positive is held out; the training fold has only the safe timestamp contract.
GAT (all seeds)	ArithmeticVulnerable, Arithmetic	FN in all seeds	The same single-example problem. The contract's slice has 8 nodes, and no node is scored above 0.3.
Sink rule	ReentrancySafe, Reentrancy	FP	The contract contains a real external call. The rule does not consider that the state is updated before the call.
Sink rule	ArithmeticVulnerable and ArithmeticSafe, Reentrancy	FP (both)	The trigger is the substring `send` in `require(balances[msg.sender] >= ...)`. This is the keyword problem in Section 8.
Sink rule	ArithmeticSafe, Arithmetic	FP	The rule flags any *, so it cannot tell a checked multiplication from an unchecked one.

Two patterns stand out. First, the GAT's held-out errors are stable across seeds: each (contract, class) pair errs in all five seeds, with one exception. The one exception is a single extra false positive for TimestampVulnerable under the Arithmetic class in one seed. This means the errors come from the data, not from initialization noise. Second, the sink rule's errors come from keywords and missing context, not from randomness.

The label fix changed localization for the Timestamp contract. Before the fix, the labels pointed at line 10, which holds no timestamp check, and the auditor reported that it could not localize the line. After the fix, the same contract is localized to line 9. The Arithmetic contract stayed unlocalized. The before and after logs are in `results/logs/before_fixes/` and `results/logs/`, and notebook 03 compares them.

7. Discussion

RQ1 (fit). The GAT learns its six training contracts perfectly and gives high confidence to the correct class. Learning the training data is therefore not in question. The question is what it learned.

RQ2 (generalization). Under leave-one-contract-out the GAT does not generalize. The reason is structural and not a matter of tuning. Each class has one positive and one negative contract, so holding out a contract removes the only positive for its class. The model then has to infer a pattern from a single example, and in our folds it infers the wrong one. In this setting the learned model

did not beat the fixed rule, which had higher recall. The sink-presence rule's precision of 0.429 is also a warning: a learned model that just reproduces the rule would add nothing.

RQ3 (localization). Localization works in-sample for the Reentrancy and Timestamp contracts. Under leave-one-out it fails in every fold, because the node scores depend on the same class signal that the classifier cannot learn from one example. The localization result therefore depends on the training set and should not be read as a property of the architecture.

What the defects show. Both code defects (the label offsets and the legacy operator lookup) survive an in-sample check, because the in-sample check reuses the same labels and graphs. The operator bug removed the only arithmetic sink from a legacy-syntax contract, so the arithmetic class had no arithmetic seed in that graph. This suggests that in-sample checks do not catch input-handling bugs, and that tests for each syntax format are needed. We did not test this beyond the one contract.

8. Limitations

1. **Six contracts.** Three positive and fifteen negative class decisions cannot support precision or recall estimates with any confidence. A single decision changes the sink rule's precision by roughly 0.07 to 0.14.
2. **Keyword features.** Flags are set by substring matches. `"now"` matches inside words such as *know*, and `"send"` matches inside `msg.sender`. The sink-rule errors above show that these substrings produce false signals. An ablation to measure their effect on the GAT has not been run.
3. **Hand-set thresholds.** The 0.5 graph and 0.3 node thresholds were not tuned on held-out data.
4. **Narrow compiler coverage.** Solc 0.7.6 (legacy AST) appears in one contract. Versions 0.4.x to 0.6.x are mapped but untested.
5. **Only one GNN variant.** The TMP and MRN-GCN variants described in the earlier draft are not implemented.
6. **No public benchmark.** SmartBugs and SolidiFI are the natural held-out sets (Durieux et al., 2020; Ghaleb & Pattabiraman, 2020). Using them requires a separate decision about downloading them, which was not made for this report.
7. **No calibration.** The reported confidences are sigmoid outputs and have not been checked against observed frequencies.

9. Conclusion

We built and tested a pipeline that turns Solidity contracts into multi-relational graphs, slices them, and classifies them with a GAT. On six demo contracts it fits its training data exactly and points to the Reentrancy call and its check. Under leave-one-contract-out evaluation it does not detect any held-out vulnerable contract, and a fixed sink-presence rule has higher recall. The pipeline is a reproducible starting point, not a detector. The next step is the held-out evaluation on

SolidiFI and SmartBugs with precision, recall and F1 reported on contracts the model has not seen, and with comparison against the tools named in Section 2.

Reproducibility and AI disclosure

The repository includes the code, the saved weights, three executed notebooks, all metrics and logs, a data card, a model card, a failure log and a reproducibility checklist. All numbers in this report come from files in `results/`. The commit and tag for the submitted version are listed in `README.md`. The project was done with an AI assistant, as disclosed in `AI_USE.md`. The author reviewed and is responsible for all content.

References

De Baets, C., Suleiman, B., Chitizadeh, A., & Razzak, I. (2024). *Vulnerability detection in smart contracts: A comprehensive survey* [Preprint]. arXiv:2407.07922.

Durieux, T., Ferreira, J. F., Abreu, R., & Cruz, P. (2020). Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE 2020)* (pp. 530–541). arXiv:1910.10601.

Feist, J., Grieco, G., & Groce, A. (2019). Slither: A static analysis framework for smart contracts. In *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB 2019)*. arXiv:1908.09878.

Ghaleb, A., & Pattabiraman, K. (2020). How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2020)*. arXiv:2005.11613.

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR 2017)*. arXiv:1609.02907.

Liu, H., Fan, Y., Feng, L., & Wei, Z. (2023). *Vulnerable smart contract function locating based on multi-relational nested graph convolutional network* [Preprint]. arXiv:2306.04479.

Luu, L., Chu, D.-H., Olickel, H., Saxena, P., & Hobor, A. (2016). Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS 2016)*. https://doi.org/10.1145/2976749.2978309

Nguyen, H. H., Nguyen, N.-M., Xie, C., Ahmadi, Z., Kudenko, D., Doan, T.-N., & Jiang, L. (2022). *MANDO: Multi-level heterogeneous graph embeddings for fine-grained detection of smart contract vulnerabilities* [Preprint]. arXiv:2208.13252.

Sayeed, S., Marco-Gisbert, H., & Caira, T. (2020). Smart contract: Attacks and protections. *IEEE Access, 8*, 24416–24427. https://doi.org/10.1109/access.2020.2970495

Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., & Alexandrov, Y. (2018). SmartCheck: Static analysis of Ethereum smart contracts. In *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB 2018)* (pp. 9–16).

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations (ICLR 2018)*. arXiv:1710.10903.

Zheng, Z., Xie, S., Dai, H.-N., Chen, W., Chen, X., Weng, J., & Imran, M. (2020). An overview on smart contracts: Challenges, advances and platforms. *Future Generation Computer Systems, 105*, 475–491. arXiv:1912.10370.

Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., & He, Q. (2020). Smart contract vulnerability detection using graph neural network. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI-20)* (pp. 3283–3290). https://doi.org/10.24963/ijcai.2020/454